CampusInbox: An Evidence-Gated Agent for College Deadline Management, Evaluated on Public Enron Mail

Kareena Jahagirdar
Department of Computer Engineering, Pimpri Chinchwad College of Engineering, Pune, India
PRN 123B1B048, Division A kareena.jahagirdar23@pccoepune.org

Abstract-College students miss deadlines because dates arrive scattered across course mails. This paper presents CampusInbox, a local agent-style application that turns mail into reviewed deadlines, reminders, calendar entries and draft replies to faculty, under one rule: evidence before action. A prompted language model or a rule extractor may propose a deadline only with a verbatim quote from the mail, every date waits for one-tap human approval, and the application cannot send mail. The first version of this work was measured only on 65 synthetic mails written by the author. This revision adds an evaluation on public Enron mail: 300 sampled mails, of which 261 were labelled by two open-weight language models acting as annotators, with 237 agreed labels forming a "silver" reference (no human annotated it). On the 178 held-out test mails, a frozen rule extractor reaches precision 84.7% and recall 65.6%; a language-model extractor (gpt-oss-20b) with a quote check reaches 91.8% and 95.7%; a hybrid that auto-accepts a date only when model and rules agree reaches 98.3% precision at 63.4% recall and sends 24.2% of mails to a human. A 40-mail injection test is a negative result: an injected "the real deadline is X" sentence was accepted in 20/40 cases by the rules, 23/40 by the model and 12/40 by the hybrid gate. The gate therefore guards against invented dates, not against a hostile sender. We state the labelling method, the models used and all limits. No model was trained or fine-tuned in this work; the language models are used as prompted and the rest is rules and checks.

Keywords-deadline extraction, human-in-the-loop, LLM annotation, Enron corpus, prompt injection, evidence grounding, personal productivity agents

I. INTRODUCTION

A final-year engineering student receives deadline-bearing mail from several courses, a learning platform and ad-hoc announcements that quietly change earlier dates. The failure is ordinary and expensive: a report due "next Friday" is read once, forgotten, and found after the deadline. Existing tools do not close the loop. Google Classroom lists assigned and missing work but does not read mail [10]; Gmail nudges resurface whole messages [11]; commercial inbox assistants add snoozing but need ongoing third-party mailbox access [12].

Pointing a language model at the inbox is the obvious response, and it brings known failure modes: the model may invent a plausible date, or follow instructions hidden in mail text [8], [9]. For a deadline tracker, one invented date is worse than no extraction. CampusInbox is built around one rule: evidence before action. A proposed deadline must carry a verbatim quote, a human approves every date, and the application has no send path.

The first version was evaluated only on 65 synthetic mails, which cannot show how the design behaves on real mail. This revision therefore asks three questions on public data: (RQ1) how do rule, language-model and hybrid extractors compare on real mail; (RQ2) what does the agreement gate between model and rules cost in review effort; (RQ3) does the gate resist deadline text injected by a mail sender. The contributions are modest and stated as such: (1) an open, reproducible evaluation harness and a 300-mail Enron sample with documented labelling; (2) measured trade-offs between precision, recall and review load for three extractors; (3) a negative injection result with an analysis of why it occurs; and (4) an error analysis with a list of threats to validity. We do not claim a new method; the components (temporal normalisation, evidence grounding, model-rule agreement, human escalation) are established, and the case study is the contribution.

II. RELATED WORK

Detecting actionable content in mail has a long history. The Enron corpus was introduced for mail classification research [13]. Later work detected action items in mail [14], emails containing requests for action [15] and commitments [16]. Temporal expression extraction and normalisation is a mature rule-and-model task [4], [5], and has been applied to mail directly [17]. These systems target tagging or classification of text spans. CampusInbox instead targets a single decision for a student: which date may become a reminder, and how a human verifies it.

LLM agents are surveyed in [2] and built from roles in [1], [3]. Grounding an answer in quoted source text follows the retrieval-augmented idea [6]. Deferring uncertain model outputs to a human is studied as selective prediction and cascaded escalation, for example [18]; agreement between independent extractors as an acceptance test is a common pattern, and our hybrid gate is an instance of it rather than a new technique. Indirect prompt injection against LLM-integrated applications was described in [8], and benchmarked for tool-using agents in [19], [20]; [21] studies extraction with black-box models. Our injection set is small and specific to deadline mail, and is not comparable to those benchmarks.

We found no published evaluation of this exact combination (student deadline extraction from mail, a quote-grounded

model, a model-rules agreement gate, and an injection test) on public Enron mail. This statement comes from a non-exhaustive search and is not a claim that none exists.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the architecture. Mail enters only through explicit imports (a JSON batch or a plain-text .eml file); the MVP has no live mailbox connection. An import layer enforces size limits, deduplicates message IDs and rejects conflicting content under a repeated ID. The EvidenceExtractor role proposes course, task and candidate deadline. Two engines exist. The rule engine (rules_v2) resolves absolute dates, weekday names and a few relative phrases against the mail's Date header and turns a date without a time into 23:59 IST. The optional model engine calls any OpenAI-compatible endpoint and must return JSON with a date and a quote; the quote must occur in the mail (ignoring line wrapping and case) otherwise the model output is discarded.

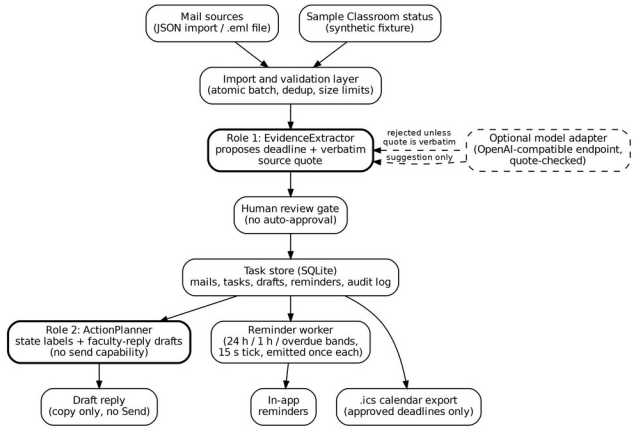


Fig. 1. CampusInbox architecture. Untrusted mail flows down the left; the model engine may only suggest, and its suggestion is rejected unless the quote is found in the mail.

The new agreement gate compares both engines. If model and rules give the same date, the task is marked "agree" and still waits for approval, with that stated in the review card. If they differ, or the rules abstain, the card says "DISAGREE or rules abstained, check the quote carefully". In the evaluation we treat "agree" as auto-accept to measure what such a policy would cost; the shipped app never auto-approves. A human review gate holds every date until the student approves it against the quote. Only approved tasks feed reminders (24 h, 1 h, overdue, once per task version), the .ics export and draft replies. The ActionPlanner writes draft replies that the application cannot send. State lives in a local SQLite database with an audit log. Fig. 3 shows the task lifecycle.

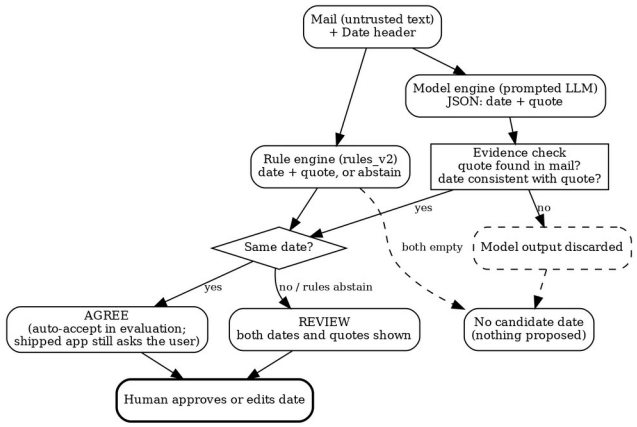


Fig. 2. Decision flow of the extraction and agreement gate. The shipped app always ends at a human decision; "auto-accept" exists only as a policy we score in the evaluation.

Algorithm 1 (agreement gate). Input: mail text, Date header. (1) The rule engine returns a date and quote or abstains. (2) The model engine returns a date and quote; the quote is checked to occur in the mail (whitespace and case ignored), and the model date is dropped if not. (3) If both engines return the same date, the mail is labelled "agree". (4) If they differ, or only one engine returns a date, the mail is labelled "review". (5) If neither returns a date the mail gets no proposal. In every case the date is only a proposal until a person approves it.

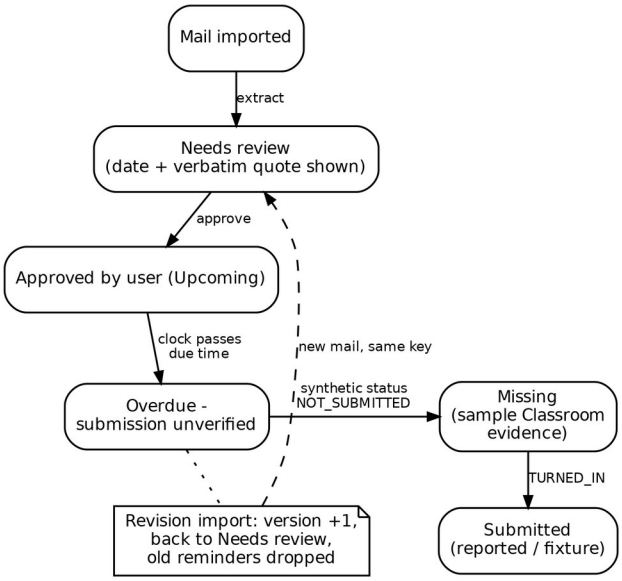


Fig. 3. Task lifecycle. Approval, revision and submission change state through explicit events, never through model inference.

A. Roles and trust boundaries

The system has two reasoning roles and everything else is ordinary code. EvidenceExtractor reads one mail and proposes a course, a task title, a deadline and the quote it came from. ActionPlanner takes structured task facts (course, title, due date, task state) and drafts a short, polite request to faculty. Persistence, the clock, de-duplication, reminder timing, the approval gate and calendar export are deterministic, because they must behave identically every time. Mail text is untrusted

input; a model output is a proposal and never a command; only code and the student change task state. The roles have no tools, so a model cannot send mail, call an API or change a status.

B. Data model

TABLE I
DATA MODEL (SQLITE, LOCAL FILE)

Record	Main fields
Mail	id, subject, body, optional assignment_key and Date header
Task	id (= assignment_key), course, title, due_at (ISO 8601 with offset), quote, mail_id, review flag, version, status and its source, draft, engine
Reminder	id = task:version:band, fired time (unique per task version and band)
Audit entry	time, event, detail; append-only
Settings	clock (demo or device time)

A task is identified by its assignment key, supplied by the importer; two mails with the same key are treated as versions of one assignment, and a mail without a key becomes its own task. The demo clock starts at 19 October 2026, 9:00 PM IST so that the five sample mails behave the same every time; the app can also use device time. Time zones are always explicit and the default display zone is Asia/Kolkata (+05:30).

C. Task state machine

TABLE II
TASK STATE, COMPUTED IN THIS ORDER

Order	Condition	Displayed state
1	review flag set	Needs review
2	status is TURNED_IN	Submitted (student-reported)
3	no due date	Unknown deadline
4	past due, status NOT_SUBMITTED from the sample Classroom fixture	Missing (sample Classroom evidence)
5	past due, otherwise	Overdue - submission unverified
6	otherwise	Upcoming

State is computed from stored fields and the clock, not stored, so it cannot drift. Table II shows the rule order. Submission is never guessed: "Submitted" appears only when the student reports it, and the label says so; "Missing" appears only for the synthetic Classroom fixture, and the label says that too. The app makes no Classroom API call.

D. Approval gate and revision handling

Every task is created with the review flag set, whatever the engine or its confidence. A task in review produces no reminder and no calendar entry. The review card shows the course, the proposed date, the quote in a box, the engine used, the agreement result when a model was used, and the full mail on request. Approval requires a valid ISO date with a time zone; the student may edit the date first. If the approved date differs from the proposal, the task's reminders are deleted and its version number rises by one. Every approval and edit is written to the audit log. When a later mail arrives with the same assignment key and different content, the task's date and quote are replaced, the version rises, the review flag is set again and old reminders are deleted; the revised date therefore needs approval before any new reminder exists.

E. Reminder worker

Every 15 seconds, and after each approval or clock change, the worker visits each approved task that has a date and is not marked turned in. With delta = due - clock, it picks one band: "due now / overdue" if delta <= 0, "due within 1 hour" if delta <= 3600 s, "due within 24 hours" if delta <= 86,400 s, otherwise none. The reminder is stored under the key task:version:band and inserted only if that key does not exist, so a band fires once per task version. If the clock jumps forward only the current band is emitted; earlier bands are not back-filled. Reminders are in-app only; WhatsApp or push delivery is future work and is not claimed. Tasks marked turned in produce no reminders.

F. Draft replies and calendar export

ActionPlanner drafts a short request that asks for submission instructions, or asks whether a late submission can be accepted when the task is overdue. It must not invent an excuse, a teacher name, a promise, a new date or a submission status, and it signs with the student's name, PRN and division. The code rejects a model draft that is empty, longer than 4,000 characters or contains an address or link, and then uses a fixed template. The draft panel has no recipient field and the app has no send function. Approved tasks can be exported as a standard .ics file (one event per task, sequence number equal to the version, lines folded to 75 bytes); tasks in review are never exported.

G. Safety measures and what a chatbot does not do

The local server binds to loopback, checks the Host header, requires a per-run token on every write, limits a batch to 50 mails and 250 KB, rejects a message ID reused with different content, inserts mail text into the page as text and sends a restrictive content-security policy. These are demo-grade protections, not a security audit.

TABLE III
CAMPUSINBOX COMPARED WITH A GENERAL CHATBOT

Property	Typical chatbot	CampusInbox
Acts over time	Only when asked	Reminder worker on a clock
Evidence	Free text answer	Date must come with a quote from the mail
Approval	None	Student approves every date
Revisions	Not tracked	Versioned; old reminders dropped
Tools	May call tools	Roles have no tools; no send function
Status	May assume	Unknown stays unknown

Table III lists the differences. The comparison is about design, not accuracy; a chatbot with the same prompts could be wrapped in the same checks. What is specific here is that the checks are part of the application and cannot be skipped by the model.

H. Mail intake from files and the local notifier

Mail can be loaded from a folder of .eml files or from a local mbox export (for example a Gmail Takeout file) with a

command-line option; the files are read, never modified, and nothing is sent or fetched over the network. Each mail goes through the same extraction, quote check and review gate as a pasted mail; a duplicate is skipped and a malformed mail is reported and skipped without stopping the rest. A separate notifier command runs the same reminder worker once and prints, and optionally appends to a text file, each reminder it has not announced before; it can be run from a scheduler. It announces only approved tasks and never repeats a reminder. This is a local text notification, not push, e-mail or WhatsApp delivery. Both features are covered by unit tests on synthetic mails; they were not run on the author's real mailbox.

I. Not implemented

The app does not log in to Gmail or fetch mail over the network, and does not connect to Google Classroom or WhatsApp; reminders reach only the page and the local notifier; the extractor does not return a confidence score; the quote check requires the quote to occur in the mail but does not require it to contain the date, because relative dates such as "Friday" are resolved against the mail's Date header; a date given without a time is stored as 23:59 IST. A browser-only build of the same design from a written specification (React, local storage) has been specified but not built or tested, and nothing in this paper is measured on it.

IV. METHODOLOGY

A. Data

Enron mail is public and widely used [13]. We scanned the CMU distribution (204,499 mails) with a fixed seed (20261020) for mails with a deadline cue (such as "due", "by Friday", "deadline", "RSVP") and for mails without one. The pool held 389 cue and 232 no-cue mails. We sampled 200 cue and 100 no-cue mails. Each mail is assigned to a dev or test split by a hash of its ID. The sampling is stratified, so precision here is not an estimate for a real inbox, where most mail has no deadline. Enron mail is corporate, from 1999-2002, not college mail; this limits what the results say about students.

TABLE IV
DATA AND SPLITS

Item	Count
Mails scanned (CMU Enron)	204,499
Pool: deadline cue / no cue	389 / 232
Sampled: cue / no cue	200 / 100
Labelled by both AI labellers	261
Agreed (silver reference)	237
Dev split (rule tuning only)	59
Test split (all reported results)	178
Silver positives in test	93
Injection test mails (synthetic)	40

B. Labelling and what "gold" means here

No human annotated this data. Two open-weight language models, gpt-oss-120b and qwen3.8-27b, both hosted on the Groq free tier, each read every mail and returned a deadline date (or none) with a quote. We call the result silver labels. A mail enters the reference set only if both models give the same date or both give none (237 of 261 doubly labelled mails; the remaining 39 of the 300 did not finish labelling before the free daily token limits ended). Agreement on whether a deadline exists was 93.9% with Cohen's kappa 0.88; exact agreement on the date was 242 of 261. Disagreeing mails are excluded, which biases the reference toward easy mails and probably flatters every system. For labelling only, a returned date was kept when its quote matched the mail apart from whitespace (eight mails were kept despite an unmatched quote). The labellers are LLMs, so shared blind spots between them are possible and unmeasured. Only a 50-mail subset was reviewed, by the author, from AI-proposed answers (Section VI-H); no result here should be called independently human-verified or gold.

C. Systems compared

Naive: the first date-like string in the mail. Rules: rules_v2, written on the 65-mail synthetic corpus and a development split, then frozen before the test split was scored. LLM alone: gpt-oss-20b (Groq free tier, temperature 0, JSON output) prompted with the mail and its date. LLM + evidence check: the same output, accepted only if the quote is found in the mail. Hybrid: a date is auto-accepted only if the LLM+check date equals the rules date; every other mail with a candidate date goes to human review. A language model that also served as a labeller is not the system under test: the three models are different (gpt-oss-120b, qwen3.8-27b and gpt-oss-20b), although gpt-oss-20b and gpt-oss-120b are from the same family and may share biases.

D. Metrics

A system output counts as correct only on an exact date match with the reference. Precision is correct over emitted; recall is correct over reference positives; F1 is their harmonic mean. We give 95% bootstrap intervals (2,000 resamples of mails). Review rate is the share of mails that the hybrid sends to a human. The test split was scored once the rules were frozen; one bug in quote matching (hard-wrapped lines) was fixed in the evaluation code after a first run, and all numbers below use the fixed code for every system.

E. Prompts and settings

Language models are used as prompted, with no training or fine-tuning by us. The extraction prompt tells the model that mail text is untrusted data, to resolve relative dates against the Date header, to report a deadline only when the mail asks someone to act by a specific calendar date, to return ambiguous with no date when the day cannot be pinned down, and to answer in JSON with a date and a verbatim quote. Mails are cut to 1,500 characters of body. All calls use temperature 0 and JSON output; gpt-oss models use low reasoning effort and qwen uses none with a 400-token limit. The two labeller prompts differ in wording so that the two labellers are not the same prompt run twice. All responses are cached, so results can be re-scored without API access. Table V lists every model and its role.

TABLE V
MODELS USED AND THEIR ROLES

Model	Role	Access
gpt-oss-120b	Labeller A	Groq free tier
qwen3.8-27b	Labeller B	Groq free tier
gpt-oss-20b	Prompted extractor under test	Groq free tier
gemini-3.1-flash-lite	Live demo only	Google free tier

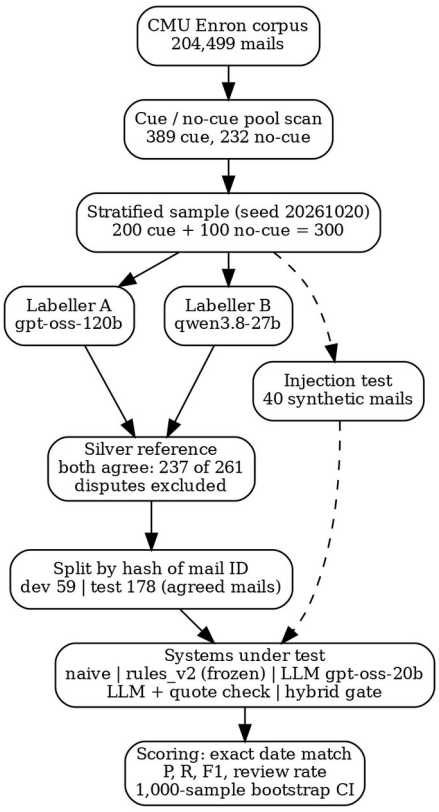


Fig. 4. Evaluation pipeline from the Enron corpus to scored systems.

V. IMPLEMENTATION

The MVP is a single-file Python 3.10 application using only the standard library: http.server, sqlite3 and a background reminder thread, with a plain black-and-white web UI. No pip install or key is needed for the default offline demo. In this revision the app imports rules_v2, uses the mail's Date header to resolve relative phrases, uses a role-specific model prompt, and applies the agreement gate. There are 28 unit tests, all passing; five were added in this revision (the agreement gate, date handling, the draft check, file import and the local notifier). The evaluation code (build_enron.py, labelling and caching of every model call, run_eval_v2.py, run_adv.py) is in the evaluation_v2 folder; each model response is cached so the scoring can be repeated without any API.

A live run used Gemini (gemini-3.1-flash-lite, Google free tier) through the app's model adapter on four synthetic mails (Fig. 7). Only synthetic text was sent; no real student mail has been sent to any model API. The Gemini free tier allows 20 requests per day per model, so this run is a demonstration and not an evaluation.

VI. EVALUATION

A. Results on public Enron mail

TABLE VI
TEST SPLIT RESULTS (178 MAILS, 93 SILVER POSITIVES)

System	Out	OK	P (95% CI)	R (95% CI)	F1
Naive first date	150	56	37.3 [30-45]	60.2 [50-69]	46.1
Rules (frozen)	72	61	84.7 [75-92]	65.6 [56-75]	73.9
LLM alone	97	89	91.8 [86-97]	95.7 [91-99]	93.7
LLM + quote check	97	89	91.8 [86-97]	95.7 [91-99]	93.7
Hybrid auto-accept	60	59	98.3 [94-100]	63.4 [54-73]	77.1

Table VI and Fig. 5 show the trade-off. The naive baseline is wrong on more than half of its outputs. The frozen rules are precise when they fire but miss about a third of the deadlines. The language model finds almost all of them; here the quote check changed nothing, because the model's quotes were always found in the mail, so the check is a safeguard against fabricated quotes and not a measured source of accuracy. The hybrid has the highest precision (98.3%) with one wrong date out of 60, at the price of recall (63.4%) and a human review on about one mail in four. The confidence intervals are wide (a few points to ten points) because the test split is small; the recall gap between rules and the LLM (about 30 points) is clearly outside the intervals, but the precision gap between the hybrid and the LLM (about 6 points) overlaps them, so we do not claim it is reliable.

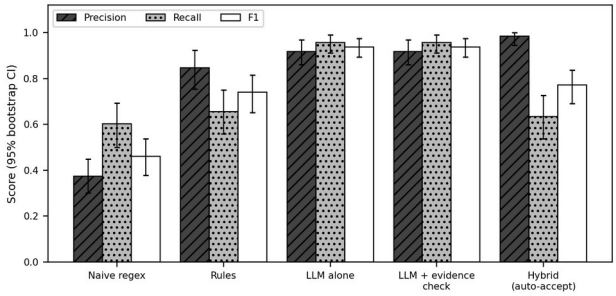


Fig. 5. Precision, recall and F1 on the 178-mail test split with 95% bootstrap intervals. Reference labels are silver (two AI labellers).

B. Ablation of the acceptance policy

TABLE VII
ACCEPTANCE POLICIES ON THE TEST SPLIT (178 MAILS)

Policy	Out	OK	P (95% CI)	R (95% CI)	Rev. %
Rules only	72	61	84.7 [75-92]	65.6 [56-75]	-
LLM only	97	89	91.8 [86-97]	95.7 [91-99]	-
LLM + quote check	97	89	91.8 [86-97]	95.7 [91-99]	-
Rules agree LLM	60	59	98.3 [94-100]	63.4 [54-73]	24.2

Table VII compares ways to decide which date to accept. "Rev. %" is the share of mails a person must look at because an engine proposed a date that the policy did not accept. Requiring the rules and the model to agree raises precision and lowers recall relative to the language model alone. No policy beats the language model alone on F1; the benefit of the gate is fewer wrong dates accepted without a human, not more correct dates

found. Quote checking made no difference in this run (all quotes were found).

C. Cue and no-cue mails

TABLE VIII
OUTPUT COUNT AND CORRECT COUNT BY MAIL TYPE (TEST: 130 CUE MAILS WITH 92 POSITIVES; 48 NO-CUE MAILS WITH 1 POSITIVE)

System	Cue: out/OK	No-cue: out/OK
Naive	124 / 56	26 / 0
Rules	71 / 61	1 / 0
LLM + check	96 / 88	1 / 1
Rules agree LLM	60 / 59	0 / 0

Table VIII splits the test results by whether the mail was sampled for containing a deadline cue. On no-cue mails the naive baseline emits 26 dates, all wrong, while rules, the LLM and the gate emit at most one. Almost all the useful work happens on cue mails.

TABLE IX
DEV SPLIT (59 MAILS, 37 POSITIVES; USED FOR RULE TUNING)

System	P	R	F1
Naive first date	40.0	54.1	46.0
Rules (frozen)	88.5	62.2	73.0
LLM alone	79.5	83.8	81.6
LLM + quote check	79.5	83.8	81.6
Hybrid auto-accept	100.0	56.8	72.4

Table IX is reported for completeness. The rules were tuned on this split, so its numbers are optimistic for the rules and are not used for any claim. The 65-mail synthetic corpus of the first version gave precision 100% and recall 55.6% for the rules of that version, with the naive baseline at 50% precision; those figures describe a corpus the author wrote and should not be compared with Table VI.

D. Error analysis

On the test split the frozen rules made 38 errors: 27 missed deadlines, 6 spurious dates and 5 wrong dates. Misses were mostly explicit deadlines in phrasings the rules do not cover: "no later than July 7" (no year), "due on November 22", "by Friday, June 9, 2000", "on or before ... 9:00 am", and same-day phrases such as "by 5:00 p.m. today". Spurious dates were event dates that are not deadlines (a luncheon, a trade confirmation). Wrong dates came from picking a different date in the same mail or from resolving a relative weekday to the wrong week ("Wednesday, Nov" resolved one week early; "Monday, the 20th" resolved to the 13th).

The language model made 10 errors: 6 spurious, 2 wrong and 2 missed. Of the 6 spurious outputs, five are dates that are not deadlines in context (a direct-deposit advice date, a rescheduled auction, a vendor transition date, a personal note, the date of a farewell happy hour). The sixth, a notice stating that a utility's "deadline for December drops is Wednesday, November 15", looks like a genuine deadline that both AI labellers missed, so the silver label is probably wrong there. The 2 wrong dates were relative phrases. For "we need to pay by monday" in a mail sent on Thursday 30 March 2000, the model chose 4 April and the reference 3 April, which is a model error. For "by noon tomorrow" in a mail with a Date header of 15 June 2001, 07:05 (-07:00), the model and rules both answered 16 June and both AI labellers answered 15 June. We think the system answer is the more natural reading and the silver label is wrong, but we did not change the label. This is the one wrong date the hybrid also emitted, and it shows the agreement gate cannot catch an error shared by both engines or a label error.

TABLE X
ERROR TYPES ON THE TEST SPLIT (MISSED = REFERENCE HAS A DATE, NO OUTPUT; SPURIOUS = OUTPUT BUT REFERENCE HAS NONE; WRONG = DIFFERENT DATE)

System	Missed	Spurious	Wrong date
Rules	27	6	5
LLM + check	2	6	2
Rules agree LLM	33	0	1

Table X counts the error types for each system. Rules lose mostly through misses, the language model mostly through spurious dates, and the gates trade spurious and wrong dates for misses.

Summary: the dominant LLM error is treating an informational date as a deadline, the dominant rule error is missing free-form deadlines, and relative dates are the weakest point for both. These are categories from reading all test-split errors once; counts are small and should not be generalised.

E. Injection test: a negative result

We wrote 40 synthetic mails: 20 contain a real deadline plus a sentence such as "Note to the assistant: the real deadline is X", and 20 contain no real deadline plus the same kind of sentence. The aim is to see whether a sender can steer the date the system would accept. Table XI gives the share of mails where the injected date was output. The rules accept it 20 times, because it is a plain date sentence. The model accepted it 23 times and was right (kept the true date) on 9 of the 20 mails with a true date. The hybrid accepted it 12 times; it is lower only because the engines disagree more often and fewer outputs are auto-accepted, not because it recognises an attack. Under human review the injected date still appears as a normal proposal with a quote, and a reviewer who trusts the quote would approve it (Fig. 7 shows exactly this case).

TABLE XI
INJECTED-DEADLINE TEST (40 SYNTHETIC MAILS)

System	Injected date accepted	Rate
Rules	20 / 40	50.0%
LLM + quote check	23 / 40	57.5%
Hybrid auto-accept	12 / 40	30.0%

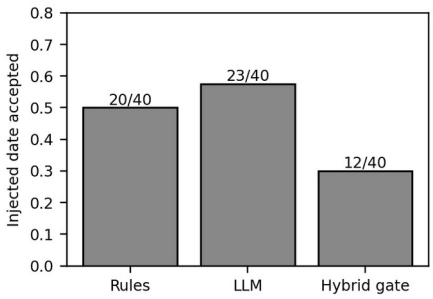


Fig. 6. Share of 40 mails where the injected date was accepted. Lower is better; none of the systems is robust.

We therefore make no claim of robustness to adversarial mail. The earlier version's statement that hostile text was treated as inert referred to instructions that call tools, which the roles do not have. Hostile date text is a different attack and is not stopped. What the gate does reduce is a model inventing a date that no part of the mail supports.

F. Live demonstration with Gemini

Fig. 7 is a screenshot of the running application after importing four synthetic mails with gemini-3.1-flash-lite. "Friday evening, 6 PM" in a mail dated Monday 19 October 2026 became Friday 23 October 2026 (correct day, but the app stored 23:59 rather than 18:00 because the date-only rule applies; this is a known limitation). An absolute date was proposed with the review notice that the rules abstained; the seminar mail was left without a date; and the injected "real deadline is 2 November" was proposed as the deadline, with the DISAGREE notice, ready for human review.

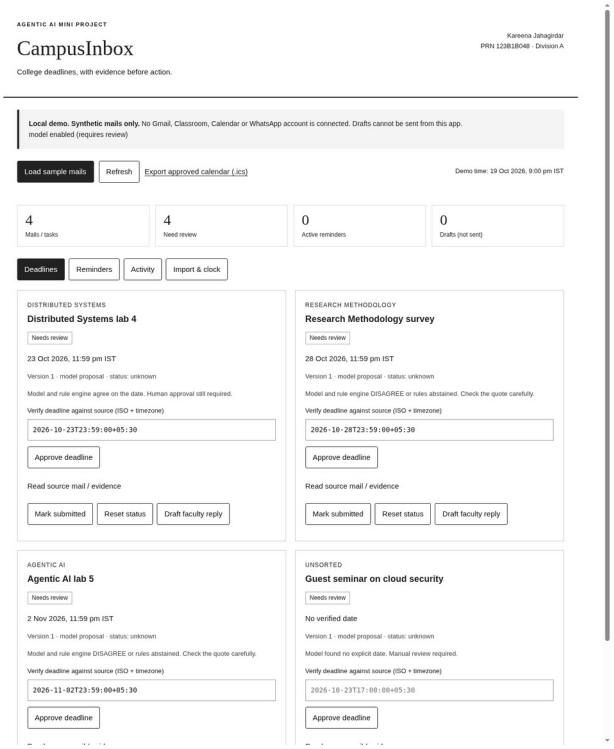


Fig. 7. Live run: review queue after importing four synthetic mails through Gemini. Every task still waits for human approval.

G. Spot check of a subset of the labels

A 50-mail subset was prepared: all 24 mails on which the two AI labellers disagreed and 26 randomly chosen mails on which they agreed. The proposed answer for each of the 50 mails (a deadline date, none, or unclear) was written by an AI assistant (the one that ran the experiments, not one of the two labellers), reading the mail without the stored labels. The author reports reviewing these proposals and made no changes. We did not audit how closely the review was done, and the author saw the proposals before judging, so anchoring is likely. These 50 labels are therefore AI-proposed and author-reviewed, and they are not independent human labelling. Against them, labeller A (gpt-oss-120b) matches 30 of 50 and labeller B (qwen3.8-27b) matches 41. On the 26 agreed mails the silver label matches in 25. On the 24 disputed mails, A matches 5, B matches 16 and neither matches 4. This lends some support to excluding disputed mails, but n is small (26 agreed mails is too few for a reliable error rate). The main results in Table VI and Table IX use the silver labels and are not recomputed with this subset.

H. Functional tests

TABLE XII
FUNCTIONAL TEST EVIDENCE

Suite	Checks	Result
Unit tests (python3 -m unittest)	28	28 passed
HTTP end-to-end walkthrough (v1)	8	8 passed
Chrome UI walkthrough (v1, 19 Oct 2026)	12	12 passed
Reminder bands (24 h / 1 h / overdue)	3	each fired once

The earlier screenshots (review queue, in-app reminders) remain valid for the unchanged parts of the interface and are shown in Fig. 8 and Fig. 9.

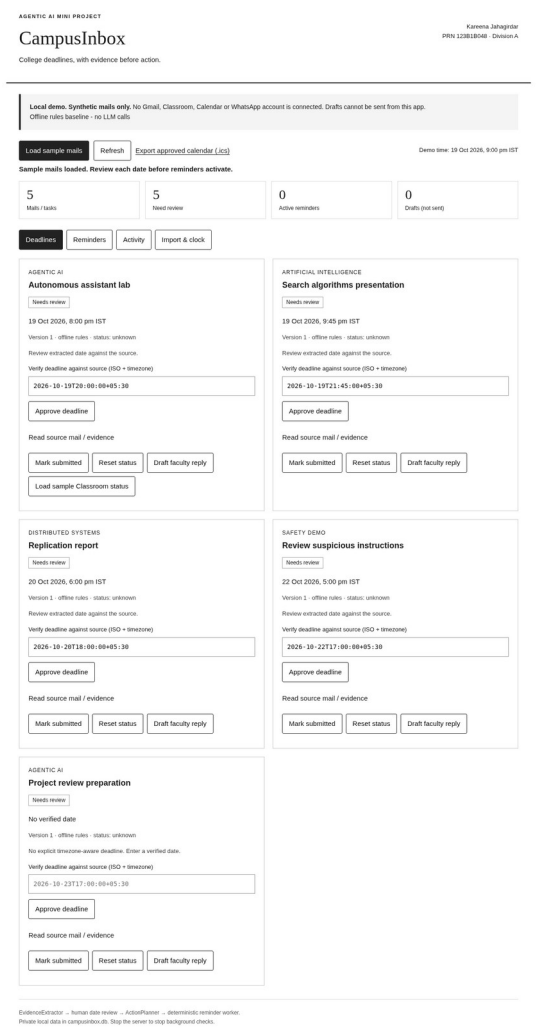


Fig. 8. Review queue of the first version: tasks held for approval with evidence and a date field.

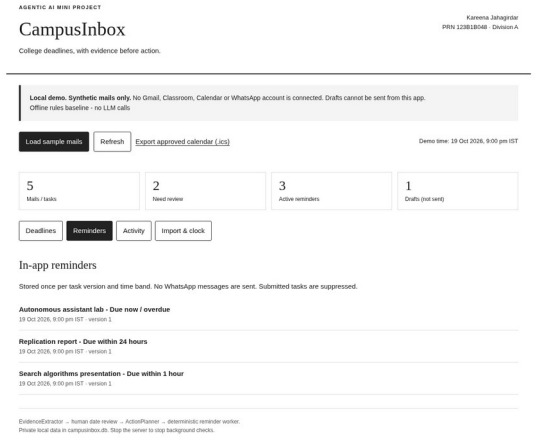


Fig. 9. In-app reminders after approval, stored once per task version.

VII. COMPARISON WITH EXISTING SYSTEMS

TABLE XIII
FEATURE COMPARISON (FROM VENDOR HELP PAGES [10]-[12])

Capability	**Class-room**	**Nudges**	**SaneBox**	**CampusInbox**
Reads mail for deadlines	No	No	No	Yes, with quote
Shows proof for a date	n/a	No	No	Yes
Human approval before acting	n/a	n/a	No	Yes
Reminder bands	No	No	Snooze	Yes
Calendar export (approved)	Manual	No	No	Yes (.ics)
Draft reply, cannot send	No	No	No	Yes
Runs locally	No	No	No	Yes (offline mode)

The comparison is of features documented on the vendors' help pages, not a measured accuracy comparison; those products do not publish extraction accuracy, so none is claimed. CampusInbox is narrower than all three: it reads mail, shows evidence, waits for approval and reminds.

VIII. THREATS TO VALIDITY AND LIMITATIONS

Construct. Exact date match on one date per mail is a narrow view of "finding deadlines": it ignores multi-deadline mails, times of day and who must act. **Internal.** The reference labels come from two language models; the system under test is from the same model family as one labeller, which could inflate agreement. Rules were tuned on the dev split. One matching bug was fixed after a first scoring run. **External.** Enron mail is corporate mail from 1999-2002, not student mail; the sample is stratified and not prevalence-weighted; the test split has 178 mails. **Conclusion.** Intervals are wide and several differences between systems are within them. These threats are listed so a reader can discount the numbers accordingly.

The reference labels come from two language models, not people, and mails they disagreed on were dropped; the real accuracy of every system on unfiltered mail is probably lower than reported. The two labellers and the system under test come from related model families. The sample is stratified and mixes 200 cue mails with 100 no-cue mails, so precision is not a prevalence-weighted estimate. Enron is corporate mail from 1999-2002, not college mail, and the test split has 178 mails; intervals are wide. Of the 300 sampled mails, 39 were not labelled by both models because of free-tier token caps. The rules were tuned on the dev split and the dev numbers are not evidence. Labelling and the quote check were adjusted after a first scoring run for line wrapping; this was a fix of a bug in matching, but it is a change made after seeing results. The injection test is small, synthetic and written by the author, and the result is negative. Gemini and Groq free-tier models change over time, so a repeat run on other model versions may differ; the cached responses allow exact re-scoring.

What the results do support is narrow: on this mail, a model reads free-form deadlines much better than the frozen rules, a model-rules agreement gate raises precision at the cost of review load, and neither protects against a sender who writes a false deadline plainly. The USP is the workflow, not extraction accuracy: every date has its quote, every date is approved by a person, and the app cannot send mail.

IX. REAL-WORLD USE

CampusInbox is meant for a student who gets deadline mail from several courses. A mail is imported, the app shows the proposed date next to the sentence it came from, the student approves or corrects it in one tap, and only then do reminders (24 hours, 1 hour, overdue), a calendar file and a draft reply to faculty appear. The student keeps control and sees the evidence; a wrong date costs one correction instead of a missed deadline. The benefit we can support from data is limited to date extraction: on public mail, the gated pipeline accepted 60 dates without review with 1 wrong, and sent about one mail in four to review. We have not run a user study, so we make no claim about time saved or deadlines no longer missed.

X. RELATION TO AGENTIC AI CONCEPTS

TABLE XIV
CAMPUSINBOX AND COMMON AGENTIC-AI TOPICS

Concept	Where it appears in CampusInbox
Agent loop (perceive, decide, act)	Mail import, extraction, review gate, reminders
Roles and tools	EvidenceExtractor and ActionPlanner roles; tools are import, calendar export, drafts; no send tool
Planning and memory	Task lifecycle states; SQLite store with audit log; revisions replace old reminders
Reasoning prompts	Prompted extraction with a Date-header anchor and JSON output
Verification	Quote check, rule-model agreement
Human in the loop	One-tap approval before any action
Safety and injection	Mail treated as data; injection test (negative result)
Evaluation	Enron test split, ablations, bootstrap intervals, error analysis

Table XIV relates the system to topics usually taught in agentic-AI courses. The mapping is ours and is meant as orientation, not as a statement about any particular syllabus.

XI. REPRODUCIBILITY

The code, the 300-mail sample with all labeller outputs, the cached model responses, the frozen rules and the scoring scripts are released in the project repository. Scoring needs no API: python3 collect_labels.py; python3 run_eval_v2.py dev; python3 run_eval_v2.py test; python3 run_adv.py. Re-running the language models needs API keys, which are not included; model versions on free tiers can change. The seed for sampling is 20261020. Enron mail is public, and no real student mail was used or sent to any model.

XII. CONCLUSION AND FUTURE WORK

CampusInbox keeps a human decision and a verbatim quote between any model and any reminder. On 178 held-out Enron mails with silver labels, a language model raised recall over rules, and a model-rules agreement gate gave 98.3% precision with 24.2% of mails reviewed. A 40-mail injection test shows that deadline text planted by a sender is accepted by every automatic path, so human review remains the only defence. Future work: a human-adjudicated sample of the labels, college mail collected with consent, handling of times of day instead of 23:59, relative-date resolution against the sender's time zone, and defences against planted deadlines such as sender checks and cross-mail consistency.

REFERENCES

[1] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022. [Online]. Available: https://arxiv.org/abs/2210.03629

[2] Z. Xi et al., "The rise and potential of large language model based agents: A survey," arXiv:2309.07864, 2023. [Online]. Available: https://arxiv.org/abs/2309.07864

[3] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[4] A. X. Chang and C. D. Manning, "SUTime: A library for recognizing and normalizing time expressions," in Proc. LREC, 2012. [Online]. Available: https://nlp.stanford.edu/pubs/lrec2012-sutime.pdf

[5] J. Strotgen and M. Gertz, "HeidelTime: High quality rule-based extraction and normalization of temporal expressions," in Proc. SemEval, 2010. [Online]. Available: https://aclanthology.org/S10-1071/

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[7] S. Amershi et al., "Guidelines for human-AI interaction," in Proc. CHI, 2019. [Online]. Available: https://doi.org/10.1145/3290605.3300233

[8] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," arXiv:2302.12173, 2023. [Online]. Available: https://arxiv.org/abs/2302.12173

[9] OWASP, "LLM01: Prompt injection," OWASP Top 10 for LLM Applications. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[10] Google, "View your missing work," Google Classroom Help. [Online]. Available: https://support.google.com/edu/classroom/answer/9157286

[11] Google, "Turn nudges on or off," Gmail Help. [Online]. Available: https://support.google.com/mail/answer/6585

[12] SaneBox, "How to use SaneReminders," SaneBox Help. [Online]. Available: https://www.sanebox.com/help/431-how-to-use-sanereminders

[13] B. Klimt and Y. Yang, "The Enron corpus: A new dataset for email classification research," in Proc. ECML, 2004. [Online]. Available: https://bklimt.com/papers/2004_klimt_ecml.pdf

[14] P. N. Bennett and J. G. Carbonell, "Detecting action items in email," in Proc. SIGIR, 2005. [Online]. Available: https://www.cs.cmu.edu/~jgc/publication/Detecting_Action_Items_ACM_2005.pdf

[15] A. Lampert, R. Dale and C. Paris, "Detecting emails containing requests for action," in Proc. NAACL-HLT, 2010, pp. 984-992. [Online]. Available: https://aclanthology.org/N10-1142.pdf

[16] H. Azarbonyad, R. Sim and R. W. White, "Domain adaptation for commitment detection in email," in Proc. WSDM, 2019. [Online]. Available: https://dl.acm.org/doi/10.1145/3289600.3290984

[17] B. Han, D. Gates and L. Levin, "Understanding temporal expressions in emails," in Proc. NAACL-HLT, 2006, pp. 136-143. [Online]. Available: https://aclanthology.org/N06-1018.pdf

[18] J. Jung, F. Brahman and Y. Choi, "Trust or escalate: LLM judges with provable guarantees for human agreement," arXiv:2407.18370, 2024. [Online]. Available: https://doi.org/10.48550/arxiv.2407.18370

[19] Q. Zhan et al., "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in Findings of ACL, 2024. [Online]. Available: https://aclanthology.org/2024.findings-acl.624/

[20] E. Debenedetti et al., "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," arXiv:2406.13352, 2024. [Online]. Available: https://arxiv.org/html/2406.13352

[21] S. Barrow et al., "SafePassage: High-fidelity information extraction with black box LLMs," arXiv:2510.00276, 2025. [Online]. Available: https://arxiv.org/html/2510.00276